To compensate for the differences between "adjacent" linear models at the fourth level, some averaging (also called smoothing) can be applied when processing a new case. Instead of using the predicted value from the terminal node, the predicted value can be "filtered" back up the tree and averaged at each node by combining the predicted value from a lower level with the predicted value from the current level. This usually improves the accuracy of predictions.

Another logic method is based on *decision rules*, which are "similar" to decision trees: after all, a decision tree can be interpreted as a collection of rules. For example, the single branch of the decision tree displayed earlier can be converted into the following rule:

if Make = Toyota **&** Model = Camry **&** Location = Jacksonville
& Body Style = LE **&** 10,000 ≤ Mileage ≤ 19,999 **&** Year = 2003,
then Sale Price = $17,350

The "if" parts of a rule (e. g., "model" = Camry) are combined logically together by the "and" (&) operator, and all the tests must be true if the rule is "to fire" (i. e., for the conclusion of the rule to be applied: *Price = $17,350*). Note that there must be several decision rules in the system (the above rule represents a single branch of a tree) and we can interpret this collection of rules as connected through the "or" operator: if one rule applies to a new case, its conclusion is taken as the predicted outcome. If two (or more) rules fire, then we can combine the conclusions of these rules to determine the final predicted outcome. The other (in some sense, opposite) problem can arise if *no* rules fire for a new case! As usual, some standard remedies exist, such as the creation of a default rule that will always fire

if 0 ≤ Mileage ≤ 999,999, **then** Sale Price = $15,000

which is the overall average price of a used car. Of course, one can question the usefulness of such a rule …

These two simple cases, when two or more rules fire or no rules fire, illustrate the point that rules can be difficult to deal with. The reason is that each rule represents a separate "piece" of knowledge and *all* the rules together operate as one system (often called a *rule-based system*). Thus, it is essential to understand the consequences of adding or dropping a rule in the system. This is important in many practical situations, where experts add their own rules (from experience) to the data-generated rules. Although dropping and adding rules in a rule-based system is not a trivial task, it is much easier to drop or add a rule than to modify an entire decision tree by cutting or adding some new branches. Hence, each method has its own advantages and disadvantages.

As mentioned earlier, classification problems have been the focus of data mining research for the last few decades, and the creation of decision rules[11] has been the most popular approach for addressing these problems. Several aspects of generating rules from data have been investigated, including:

[11] A decision rule for a classification problem is often called a *classification rule*.

- *Association rules*, which describe some regularity present in the data and can "predict" any variable (rather than just the class). For example, an associate rule may state that

  if Make = Porsche **&** Model = Carrera,
  then Location in {Jacksonville, Tampa, Los Angeles, San Francisco, San Diego}

  as Porsche Carreras are sold only at auction sites in Florida and California.

- *Rules with exception*, which extend a rule with exceptions, may refer to association rules, e. g.,

  if Make = Porsche **&** Model = Carrera, **then** Location **in** {Jacksonville, Tampa, Los Angeles, San Francisco, San Diego}, **except if** Year ≤ 1997, **then** Location **in** {Austin, Houston, Dallas}

  which states that older Porsche Carreras (produced in 1997 or earlier) are sent to auction sites in Texas; or to a classification rule, e. g.,

  if Make = Toyota **&** Model = Camry **&** Location = Jacksonville **&** Body Style = LE **&** 10,000 ≤ Mileage ≤ 19,999 **&** Year = 2003, **then** Sale Price = $17,350, **except if** Color = Red, **then** Sale Price = $18,450

  as red was a rare (but popular) color for Toyota Camry cars in 2003 and that increases the price.

Rule-based systems, which consist of a collection of rules and an inference system,[12] are quite popular, because each rule specifies a small piece of knowledge and people are good at handling small pieces of knowledge! Separate rules can be discovered from data mining activities or interviewing experts, and instead of specifying the overall model only the decision rules and inference system are needed. Note that the rule-based system will try to behave like an expert, performing some reasoning on the basis of the knowledge present in the system.

5.2.4 Modern Heuristic Methods

As indicated earlier, a few prediction methods fall into the category of "modern heuristics"; these include fuzzy systems, neural networks, genetic programming, and agent-based systems. These methods originated in different research communities, and their "mechanics" are very different to classic methods such as statistics and machine learning. Because these prediction methods are of growing importance, we will discuss them in detail in Chaps. 7–9.

[12] An *inference system* is responsible for putting the decision rules in the appropriate order and combining the outcomes of the rules that fired. It may also contain strategies and controls that are typically used by experts.

5.2.5 Additional Considerations

Many other considerations must be taken into account when selecting the "best" prediction method for an Adaptive Business Intelligence system. Although the prediction error is quite possibly *the most* important measure, it only provides one dimension of a model's quality. For real-world business problems, many other factors must be considered, such as:

- *Response time.* This is an essential consideration, as any Adaptive Business Intelligence system would have a defined response time. Fraud detection systems, for example, process millions of transactions per second, so the frequency of predictions (i. e., classifications of "fraudulent" or "legitimate") is very high. Other prediction methods, on the other hand, might be used on a weekly basis (e. g., inventory management) and so the response time is not that critical.
- *Editing.* Some prediction models are difficult to edit (e. g., neural networks), while others (e. g., rule-based systems) are easy. The ability to edit a model is an important consideration, as it might be necessary to add the knowledge of experts to the final model.
- *Prediction justification.* This is an often-overlooked aspect of evaluating the usefulness of a prediction model. For some applications (e. g., credit scoring) it is very important to justify the prediction; in some cases, this might even be required by law (e. g., justification for rejecting a loan application).
- *Model compactness.* A prediction model should not be exceedingly large and complex, as that would make it difficult for humans to understand; also, it might take a longer amount of time to make predictions. According to the principle of "Ockham's Razor," a more compact prediction model is preferable over a sprawling prediction model assuming they both do an equally good job of predicting.
- *Tolerance for noise.* All prediction methods require some approach for handling missing values (e. g., the mileage of an off-lease car has not been recorded), but some methods do a better job of handling missing values than others. Also, some values might be present, but noisy (i. e., imprecise) – like stating that the color of a car is "dark" …

Because of these many factors, it may be difficult to select "the best" prediction method for the problem at hand. Different prediction methods have different properties, and so some of them may perform better or worse when trained on different data sets. Hence, it might be worthwhile to use a few methods to build a few models, and then use all the models to reach a consensus. We will explore this *hybrid systems* approach to prediction in Sect. 10.1.

5.3 Evaluation of Models

Although it is possible to use a variety of different prediction methods to build a variety of different prediction models, the key issue is which method should be applied to a particular problem. To answer this question, it is necessary to evaluate and compare different models. Because the comparisons have to be unbiased, the evaluation methodology should be fair and just. At first blush, this may seem easy. After all, after we complete and train a few models, we can test them on the data and measure the prediction error. The best model would then be selected for implementation.

Unfortunately, it is not that simple.

First of all, the amount of available data might not be that large. Even in the car distribution example, if we take into account all the different makes and models sold at all the different auction sites, our "large" data set of three million cases is actually quite small. With 20 makes that each have 10 models, sold at 50 different locations, we are talking about 10,000 variables. Our data set of three million cases suddenly turns into just a few hundred cases for each make/model at each location. Also, because some makes/models are common while others are rare, the rare makes/models will only have a few cases per location sold at irregular intervals during the past three years.

Secondly, the performance of a prediction model on the training data might be very different from the performance of the same model on an independent set of data. This might be due to overfitting, which is a common phenomenon. In short, a model tunes itself during the training stage to such an extent that all predictions on the training data set are perfect! However, the point is to develop a prediction model that performs well on *new* data! Overfitting is a serious problem, as most prediction methods are capable of generating 100% accurate predictions on the training data set.

Thirdly, prediction models that provide different outcomes require different techniques for error measurement. For instance, a prediction model may indicate whether a new case belongs to class A or B, or, if we have a larger number of classes, the probability that a new case belongs to each class. Alternately, a prediction model may predict a number (e. g., sale price) or a sequence of numbers (e. g., sale price *and* sale date). In each of these examples, we have to carefully consider what we are predicting (i. e., what the outcome of the prediction model is), and apply the appropriate error measurement technique.

Finally, we have to take into account the cost of a potential error. When classifying cases into two categories ("yes" or "no," "fraudulent" or "legitimate," etc.), there are two types of errors: (a) *false-positive*, where the outcome is incorrectly predicted as "yes," when in fact it is "no," and (b) *false-negative*, where the outcome is incorrectly predicted as "no," when in fact it is "yes" (this issue is discussed in more detail in Sect. 12.5). Clearly, the cost of these errors is very different. By classifying a legitimate transaction as fraudulent (false-positive), there is a small cost to check the transaction. On the other hand, classifying a fraudulent transaction as legitimate (false-negative) usually carries a much higher cost, especially if the transaction is significant (e. g., approving a fraudulent transaction for $5,000).[13]

Because different models may generate a different number of false-positives and false-negatives on test data, the costs of these two types of errors must be taken into

[13] A more detailed example of fraud detection is provided in Sect. 12.5.

account. Although many error measurement techniques exist (e. g., mean-squared error, mean absolute error, relative squared error, relative absolute error), it is much harder to measure the consequences of an error. For example, the error in a price prediction of a used car for a particular location might only be $150 (approximately 1% of the car's value), but this error may influence the distribution decision, which in turn influences the transportation decision and distributions of other cars (because of the volume effect)!

Because we are interested in the *future* performance of a prediction model – i. e., performance on *new* data, not performance on the training data – we cannot take a model's performance (or error rate) on the training data (i. e., *old* data) as a foolproof indicator of its performance on new data. The reason for this is very simple: The most "reliable" prediction model would be a simple lookup table where all the previous cases are stored. Such a model will score exceptionally well on old cases … Unfortunately, this score will tell us very little about the model's performance on new data! Most prediction models can be overtrained in the sense that they would behave in a similar way to a lookup table. Hence, a model's performance on the training data set will always be better than the model's true performance …

To predict a model's performance on new data, we need another data set (usually called a *test set*) that did not participate in the building, training, and tuning of the model. This is important: we need *fresh* data to evaluate the performance of a prediction model. The most popular way of doing this (when there is enough data) is to randomly divide the original data set (i. e., available cases) into a training set and testing set. The prediction method then uses the training set to select variables, compose additional variables, calculate ratios, parameters, etc., but it does not have access to the test set. Once the prediction model is created on the basis of the training data set, it can be fairly evaluated for performance on the test data set.

In many cases, the process of building a prediction model consists of two phases: (1) constructing a model, and (2) tuning the parameters of the model. For this reason, it is also convenient to further split the training data set into two subsets: the primary training set and a *validation set* – the former for building the model, the latter for tuning its parameters. So, altogether, it is convenient to have three independent data sets (the third one being the test data set, which is used to evaluate the model's performance). Each of these three data sets should be selected independently, and each of them plays an important, independent role:

- The training data set is used for building a prediction model.
- The validation data set is used for tuning the parameters of the model (i. e., for optimizing the performance of the model).[14]
- The test data set is used to evaluate the performance of the model.

If we had plenty of data for training, plenty of data for validation, and plenty of data for evaluation, then the result should be a better model. However, if there is

[14] If several prediction models were constructed from the training data set, then the validation data set is sometimes used for selecting the best model.

only a limited amount of data, then what can be done to maximize them? Note again that the general idea is to split the data: some data (usually two thirds) are used for training (this includes validation), and some (usually one third) for testing.

The first issue to consider here is whether each subset is a "representative" sample of the entire set. For example, it may happen that the training data set has no "yellow" cars, while the test data set contains many yellow cars. If a category is missing in the training set, then the prediction model might have serious difficulties in predicting the "right" value for this category (as the "learning" process is based on data). Moreover, the evaluation of the prediction model would be biased, as all (or most) cases of the category in question (e. g., "yellow" cars) would appear only in the test data set!

Clearly, it would be beneficial to "guarantee" that the distribution of cases is uniform across all data sets. One way of approaching this problem is through *stratification*: the algorithm that splits the data into training and testing subsets ensures that the sampling is done in such a way that each category is properly represented. The other approach is repeating the training and testing phases with different data sets, and then averaging the performance of the prediction model from all the iterations. A popular statistical technique, called *cross-validation*, is often used in connection with the latter approach. In this technique, we divide the data set into some number (say k) of disjoined subsets (called *folds*). Then $k - 1$ folds are used for training and one for testing, and we can repeat this process k times, each time with a different group of folds selected for training and a different fold for testing. If $k = 3$ (i. e., the data set is partitioned into three subsets), then the technique is called *three-fold cross-validation*. It is quite common to use $k = 10$ (*10-fold cross-validation*),[15] as 10 is a reasonable number of folds to get a good estimate of the prediction error.[16]

One extreme (and, in many cases, useful) application of the cross-validation technique is when the number of folds equals the number of cases in the data set (this approach is called the *leave-one-out* approach). In a database with three million cases, there would be three million folds. Hence, we would repeat the following process three million times: A prediction model is built on a training data set of 2,999,999 cases, and the error estimate is made on the remaining single case. Then the average of all errors will give us the error estimate for the prediction model. In this technique, the greatest possible amounts of data are used for training, and, because the approach is deterministic, there is no need to repeat the process. However, the computational overhead might be too large for large data sets.

The final model evaluation technique we will mention is the *bootstrap*, which has a reputation for being one of the best techniques when the data set is very small. In the bootstrap technique, a collection of cases is selected as the training

15 *10-fold cross-validation* is often used with stratification. Stratified 10-fold cross-validation is generally held as a standard evaluation technique in cases where the amount of data are limited.

16 This estimation, however, need not be perfect, as different fold selections may give different error estimates. Thus, it is a standard procedure to repeat the cross-validation process 10 times, which results in building and testing a prediction model 100 times altogether.

set *with repetition*. Further, the number of cases in the training set is the same as the total number of cases available. By doing this, some cases will be selected more than once, while some cases will not be selected al all! It is relatively easy for a mathematician to calculate the probability of a case *not* being selected for the training set by dividing the constant *e* by 1, which equals to 0.36787944117 ≈ 0.368. This means that approximately 36.8% cases *will not* be selected, and 63.2% of cases will be selected (once or more than once).[17] If we apply the bootstrap technique to our data set of three million used-car cases, then approximately 1,896,362 cases would be selected (once or more than once) for the training data set, whereas the remaining 1,103,638 cases would constitute the test data set. As with cross-validation, the bootstrap procedure is usually repeated several times with different samples.

For a moment, let us return to the issue of time dependencies in the data set. As mentioned earlier, most real-world business problems have some time-dependent relationships within their data sets: Transactions, orders, deliveries, sales – all of these have a time stamp. And because these data sets will inevitably change, the problem lies in not knowing how they will change! Also, some data sets change very quickly (e. g., the closing prices of all stocks in the S&P 500 index), while others change very slowly (e. g., the average income in a particular region). As a matter of fact, some changes are so slow that we consider the data set to be stable, even though small changes are constantly taking place. In any case, it is important to select the appropriate sampling technique when dividing the original data into training and testing sets. It is also essential to organize the cases in such a way that all the training cases have an earlier timestamp than the testing cases. This is done so that the predictions go from “past” to “future.” In other words, we should identify a particular point of time, and take all relevant *preceding* cases for the training set and all relevant *subsequent* cases for the testing set. Note also, that the time dependencies among cases might be so strong that we should treat the data set as a time series, where all cases are kept in a sequential time order.

The inevitable changes that occur in a data set – from which we are supposed to create a prediction model – have powerful consequences. If the changes are slight, then the sampling and evaluation techniques discussed in this section would work. However, if the changes are significant (like after a major stock market crash or natural disaster), then it might necessary to build a new model altogether. Also, as we saw in Sect. 5.2, different prediction methods produce different models of varying complexity. For this reason, it might be safer to select a simpler model that has a higher degree of generality (allowing for better adaptation to small changes that occur in the data set). Another approach (which we will discuss in Sect. 10.3) would be to use an adaptability module to adjust the various parameters of the model.

[17] Because 63.2% of the cases (on average) will be selected for the training set, the method is also called the *0.632 bootstrap*.

5.4 Recommended Reading

In this chapter, we gave a general overview of many different types of prediction problems (e. g., classification, regression, time series), methods (quantitative or qualitative), and processes (data preparation, data mining, model building, deployment and evaluation). Because the ultimate goal of any prediction model is to predict the "outcome" of a new case, we also discussed a variety of prediction models based on mathematics, distance, and logic. Our discussion on prediction methods will continue in Chaps. 7–9, where we will present several modern prediction methods, including artificial neural networks, fuzzy logic, and agent-based modeling. Lastly, in Chap. 10 we will discuss the concept of using several prediction models together, along with the role of the adaptability module.

There are a variety of texts available that discuss data mining techniques. The book *Predictive Data Mining* by Sholom M. Weiss and Nitin Indurkhya (Morgan Kaufmann, San Francisco, 1998) provides an excellent high-level discussion on most of the topics presented in this chapter (e. g., preparation of data, data reduction, types of solutions), with an additional discussion on data mining and statistical methods, and several case studies.

A slightly more technical introductory text to data mining techniques is *Data Mining: Practical Machine Learning Tools and Techniques* by Ian H. Witten and Eibe Frank (Morgan Kaufmann, San Francisco, 2000). The book presents many algorithms for extracting and validating various models (e. g., decision trees, rules, linear models) from data. The book also provides Java data mining tools that the authors made available through their website.

More advanced texts include *Machine Learning and Data Mining: Methods and Applications* edited by Ryszard S. Michalski, Ivan Bratko, and Miroslav Kubat (Wiley, Chichester, 1998). This volume provides a detailed treatment of many specific topics (e. g., multi-strategy approach, inductive logic programming) as well as discussions on data mining applications in pattern recognition, design, engineering, control systems, medicine, and biology.

Further, there are texts available like *Data Mining and Knowledge Discovery with Evolutionary Algorithms* by Alex A. Freitas (Springer, Berlin, 2002), which discusses the integration of some optimization and data mining techniques.

As one of the main tasks of data mining is "prediction," it is worthwhile to check some classic texts on forecasting. One of the books we recommend is *Forecasting: Methods and Applications* by Spyros Makridakis, Steven C. Wheelwright, and Rob J. Hyndman (Wiley, Chichester, 1998). The book presents a statistical approach to forecasting: from basic forecasting tools, through time series decomposition and particular methods (e. g., exponential smoothing, regression), to judgmental forecasting.

6 Modern Optimization Techniques

> "I have frequently gained my first real insight into the character of parents by studying their children."
> *The Adventure of the Copper Beeches*
>
> "The nature of his tactics suggested his identity to me, and this physical peculiarity – he was badly bitten in a saloon-fight in Adelaide in '89 – confirmed my suspicion."
> *The Disappearance of Lady Frances Carfax*

Whether in banking, manufacturing, or retail, there is scarcely an industry where the term "optimization" does not apply. This is due to the fact that every industry strives for excellence (as there are continual pressures to reduce cost and increase efficiency) and so over the years many optimization techniques have emerged to help managers find better solutions to their business problems. The field of operations research, in particular, developed many techniques to address the complexity of scheduling people, machines, and materials. We often refer to these optimization techniques as "classic" techniques, with the best examples being linear programming, branch and bound, dynamic programming, and network flow programming.

During the last decade, however, we have witnessed the emergence of a new class of optimization techniques that people have termed "modern heuristics." These modern techniques include (among others) simulated annealing, tabu search, and evolutionary algorithms, and they are the main focus of this chapter.

6.1 Overview

Irrespective of the optimization technique used, three things always need to be specified: (1) the representation of the solution, (2) the objective, and (3) the evaluation function. Let us consider each of these in turn.

The representation of a solution will determine the search space and its size. This is an important point, because the size of the search space (i. e., the number of possible solutions to the problem) is not determined by the problem, but by its *representation*. Consequently, choosing the right search space is of paramount importance. If we do not select the correct domain to begin with, we might actually preclude ourselves from ever finding the right solution!

Once we have defined the search space, we need to decide what we are looking for. What is the objective of our problem? This is a mathematical statement of the task to be achieved. It is not a function, but an expression. For example, suppose we wanted to discover a good solution to a traveling salesman problem . The objective would be to minimize the total distance of the route while satisfying the problem constraints. After the objective has been clearly defined, the next thing to do is create an *evaluation function* that allows us to compare the quality of different solutions. Some evaluation functions produce a ranking for various solutions (called *ordinal* evaluation functions), while others are *numeric* and provide a ranking and a quality measure score as well.

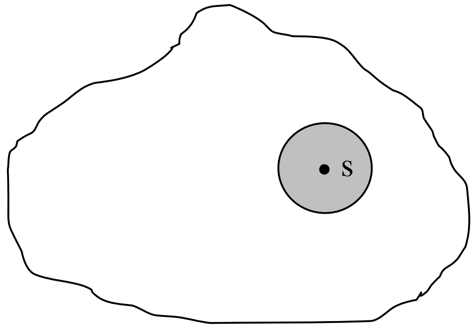
In the traveling salesman problem, a numeric evaluation function might map each solution to a distance. By comparing the distance of various possible solutions, we can easily tell if one solution is better than another and by *how much*. However, it might be computationally expensive to calculate the exact distance of each particular solution. In such cases, it might only be necessary to know approximately how good or bad a solution is, or if it compares favorably or unfavorably with some other solution. Such an ordinal evaluation function might evaluate two possible solutions and merely give us an indication as to which solution is favored.

Because the evaluation function is not provided with a problem, how should we go about choosing the correct evaluation function? Oftentimes, the objective can suggest a particular evaluation function. In the traveling salesman problem, for instance, we considered using distance as the evaluation function. This corresponds to the objective of minimizing the total distance of the route. Hence, the objective naturally suggests an evaluation function for finding the best solution. When designing the evaluation function, it is also important to keep in mind that most of the solutions we are interested in will be in a small subset of the search space (because we are only interested in *feasible* solutions – i. e., solutions that satisfy the problem-specific constraints).

Once all of these steps are complete, we can begin searching for a solution. Note, however, that the optimization technique[18] does not know what problem we are trying to solve! All it "knows" is the representation of the solution and the evaluation function. If our evaluation function does not correspond to the objective, then we will be searching for the right answer to the wrong problem!

In any search space, the goal is to find a solution that is feasible *and* better than any other solution present in the entire search space. The solution that satisfies these two conditions is called a *global optimum*. Because finding a global optimum is extremely difficult, a much easier approach is to find the best solution in a subset of the search space.[19] If we can concentrate on a region of the search space that is "near" some particular solution, we can describe this as looking at the *neighborhood* of that solution. Graphically, let us consider some abstract search space with a single solution **s**:

[18] *Optimization technique* and *search technique* are considered synonymous. The search for the best feasible solution is both an optimization problem and a search problem.
[19] This observation forms the fundamental basis of many optimization techniques.



Our intuition might tell us that solution **s** is in a neighborhood of the search space where all solutions are very similar to one another. Consequently, we can use a "neighborhood" or "local" optimization technique to find the best solution in this neighborhood. The sequence of solutions that these techniques generate while searching for the best possible solution relies on *local* information at each step of the way.

Local optimization techniques present an interesting trade-off between the size of the neighborhood and the efficiency of the search. If the size of the neighborhood is relatively small, then the algorithm may be able to search the entire neighborhood quickly. Only a few potential solutions may have to be evaluated before a decision is made on which new solution should be considered next. However, such a small neighborhood increases the chance of becoming trapped in a local optimum! This suggests using large neighborhoods, as a larger range of visibility makes it easier for the algorithm to decide where to search next. In particular, if the visibility were unrestricted (i. e., the size of the neighborhood were the same as the size of the whole search space), then eventually we would find the best series of steps to take. However, the number of evaluations might become overwhelming and impossible to compute.

All optimization techniques (whether local optimization techniques, ant systems, or evolutionary algorithms) generate new solutions from existing solutions. The main difference between these different techniques lies in how these new solutions are generated. Because we can only sample a small fraction of the search space (otherwise the computation time would be billions of years!), we should be economical in the process of generating and evaluating new solutions.

To put some of these concepts into context, let us return to the car distribution example and assume that we want to distribute 3,000 cars to 50 auction sites. Clearly, many possibilities exist for representing a "solution." For example, we can assign an index number from 1 to 50 for each auction site, and a solution can be a vector of 3,000 numbers: the first number represents the destination of the first car, the second number represents the destination of the second car, and so forth:

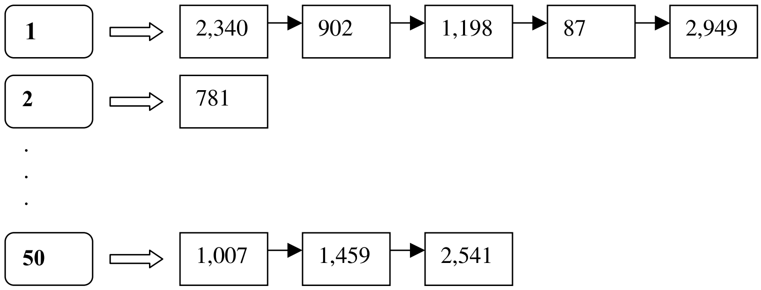
23	41	5	…	19	41

The above vector represents a solution where the first car is shipped to auction site 23, the second car is shipped to auction site 41, the third car is shipped to auction site 5, and so on, with the last two cars being shipped to auction sites 19 and 41 respectively. Of course, the auction numbers should not be assigned randomly. When we discuss some optimization techniques in the following sections, the advantages of assigning "close" numbers to "close" auctions will become clear.

Note, however, that representing a solution in this way has a couple of disadvantages. First of all, this representation implies an enormous search space that is too time consuming to search. We have 50 possible destinations for each car, so the number of possible distributions for 3,000 cars is $50 \times 50 \times 50 \times \ldots \times 50$ (i. e., 50 multiplied by itself 3,000 times!). The size of this search space can be reduced significantly by using a different representation.

The second disadvantage of this representation is that it makes some constraint handling difficult. Recall that the car distribution problem includes many soft and hard constraints, such as inventory level limits, exclusion conditions (e. g., "the total transportation distance for each car must not exceed 700 miles"), and so forth. If the above vector of auction indices were used to represent the solution, then many randomly generated solutions would be infeasible. We could reject these infeasible solutions, lower their quality measure score, or attempt to "repair" them by replacing some values in the vector with new ones. For example, if the auction site for the second car is 41 and it corresponds to Jacksonville, Florida (which is more than 700 miles from the current location of the car), then we could try to replace auction 41 with some other auction site that is closer to the location of the car. Note also that most new solutions would be infeasible with this representation, making the search process less efficient. In this particular case, other representations exist that can make constraint handling easier.

Clearly, many other representations are possible; for instance, we can create a linked-list structure of 50 nodes, where each node represents an auction site and has a list of cars "assigned" to this auction. With this representation, it would be much easier to handle certain constraints (e. g., inventory constraints, as the length of each node implies the number of cars assigned to that particular auction):



The above vector represents a solution that would send cars 2,340, 902, 1,198, 87, and 2,949 to auction 1, car 781 to auction 2, and so on, with cars 1,007, 1,459, and 2,541 going to auction 50. By using this type of representation, the size of the search space can be significantly reduced by imposing some inventory limits (e. g., each auction site should have at least 20, but no more than 100 cars). Additionally, if some auction sites do not admit cars of a particular type (e. g., high mileage cars), then it would be much easier to check (or enforce) such constraints.

Another possibility is based on indirect representation and some preprocessing. Here we would sort all the available auction sites by *distance* from a particular car, i. e., auction 1 would be the closest (distance-wise), auction 2 would be the second closest, and so forth. Although this representation looks very similar to our first representation, the interpretation is very different:

1	3	1	…	1	2

This vector represents a solution that ships the first car to the closest auction site, the second car to the third-closest auction site, etc. Note that the *same* numbers in the above representation (e. g., number 1) correspond to *different* auction sites! Again, there are several advantages of using this representation. First, the vector:

1	1	1	…	1	1

represents a solution where *each* car is sent to the closest auction site. If we believe that transportation costs play a major role in the decision-making process, then the above solution may represent a reasonable "first draft." Second, the numbers are meaningful in the sense that they correspond to distances. If for some reason auction 5 is not available (e. g., because of inventory limits), then we can direct the car to auction 6, thereby increasing the transportation distance only slightly. The third advantage is that preprocessing can help us handle many constraints. For example, if a car is red and auction 13 does not admit red cars, then we can eliminate 13 from the list of available auction sites for this car. Also, if we limit the transportation distance for any car, all we have to do is truncate the auction list to eliminate those sites that exceed the threshold. By doing this, many constraints (e. g., exclusions based on mileage, color, distance) can be handled during the preprocessing stage![20]

Note again, that the representation of a solution will define the search space and its size, and that we can define a neighborhood for any solution in any representation. If we assume a solution is represented by a vector of 3,000 numbers, with each number corresponding to an auction site, then for a solution:

23	41	5	…	19	41

we may define its neighborhood as a collection of all solutions that are identical except for one auction site being different by one (e. g., 23 can be replaced by either 22 or 24). Hence, the following solution:

23	41	6	…	19	41

is a neighbor of the original solution. Note that the size of the neighborhood is 6,000 solutions, as there are two possible replacements for each auction (23 can be replaced by 22 or 24; 41 can be replaced by 40 or 42; etc.).[21]

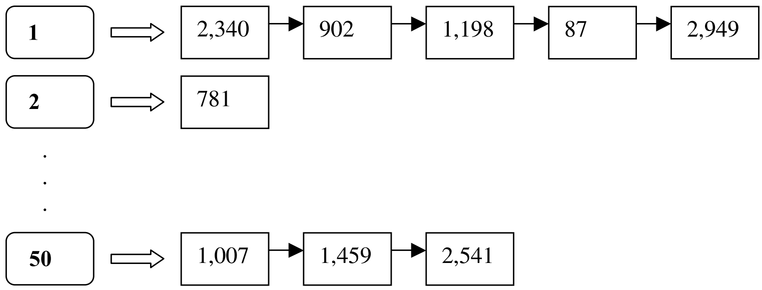
Of course, there are many other possibilities.[22] For example, if an auction site were allowed to differ by five (rather than just one), then the neighborhood would be much larger. Each auction site would define 10 possible neighbors (e. g., auction 23 could be replaced by any of the following auctions: 18, 19, 20, 21, 22, 24, 25, 26, 27, 28), so the size of the neighborhood would be 30,000. Alternatively, we can stick to the requirement that an auction site can only differ by one, but relax the restriction on the number of auction sites that can differ! In such a scenario, if any auction site can differ by one (or stay as it was), the size of such a neighborhood would be $3 \times 3 \times 3 \times \ldots \times 3$ (3,000 multiplications!) Of course, if we allow bigger changes (e. g., replacing auction 5 with auction 19), then the size of this huge neighborhood would grow even further!

[20] Using this representation, we have to build a list of all *feasible* auctions for each car. Although this preprocessing might be computationally expensive, we do it only once, at the beginning of the search. The general rule of thumb is that preprocessing is useful: the more sweat during exercise, the less blood during combat! Also, we will return to the subject of constraint handling in Sect. 6.6.

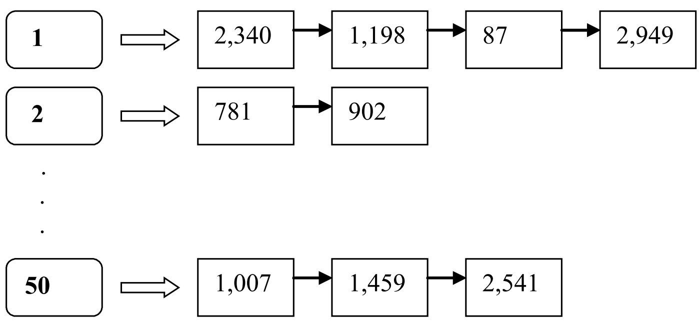
[21] For some vectors, however, the neighborhood size is slightly less than 6,000 (e.g., auction sites 1 and 50 can only be replaced by 2 and 49 respectively).

[22] The typical methods for defining neighborhoods are either based on distance or on some transformation operator.

The linked-list representation offers another possibility. For a solution:



we can define a neighbor as a new solution derived by changing the destination of one car. For example, if we move car 902 from the first auction site to the second, then we would get the following neighboring solution:



In this scenario, the size of the neighborhood is much smaller than in the previous example: there are 49 “other” auctions available for each car, so the number of neighbors is only 147,000 (i. e., 49 × 3,000). Again, we can change the size of the neighborhood by allowing some other transformations. For example, if we define a neighbor as a solution obtained by swapping the assignment of two cars (e. g., swapping the assignment of cars 87 and 1,007), then the number of possible neighbors would be less than 4,498,500 (i. e., 3,000 × 2,999/2), as swapping the assignment of some cars (e. g., cars 2,340 and 87) does not lead to a new solution.

During different stages of different optimization techniques, it is necessary to compare two different solutions and determine the better one. Hence, we must be able to evaluate any solution and assign a quality measure score to it. If the quality measure score is 123.76 for one solution and 119.92 for another, then we would like to assume that the former solution is better. In the car distribution example, it is necessary to build an evaluation procedure that returns a quality measure score for any solution. However, this task is not trivial; for example, how can we evaluate a solution:

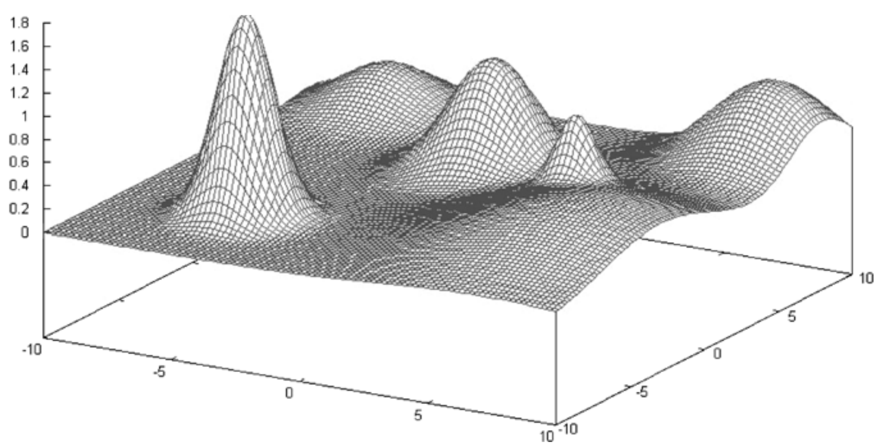
23	41	5	…	19	41

that assigns the first car to auction 23, the second car to the auction 41, and so on? Clearly, several things must be considered: the predicted sale prices of these cars at the auction sites (taking into account the time delay caused by transportation), transportation costs, "penalties" for violation of various constraints (e. g., a red car is shipped to an auction that does not admit red cars), and so forth. Quite often, there would be many trade-offs to consider (e. g., by sending a red car to a particular auction site we would violate a constraint, but on the other hand we would save a lot on the transportation cost …), which should be reflected in the evaluation function.

6.2 Local Optimization Techniques

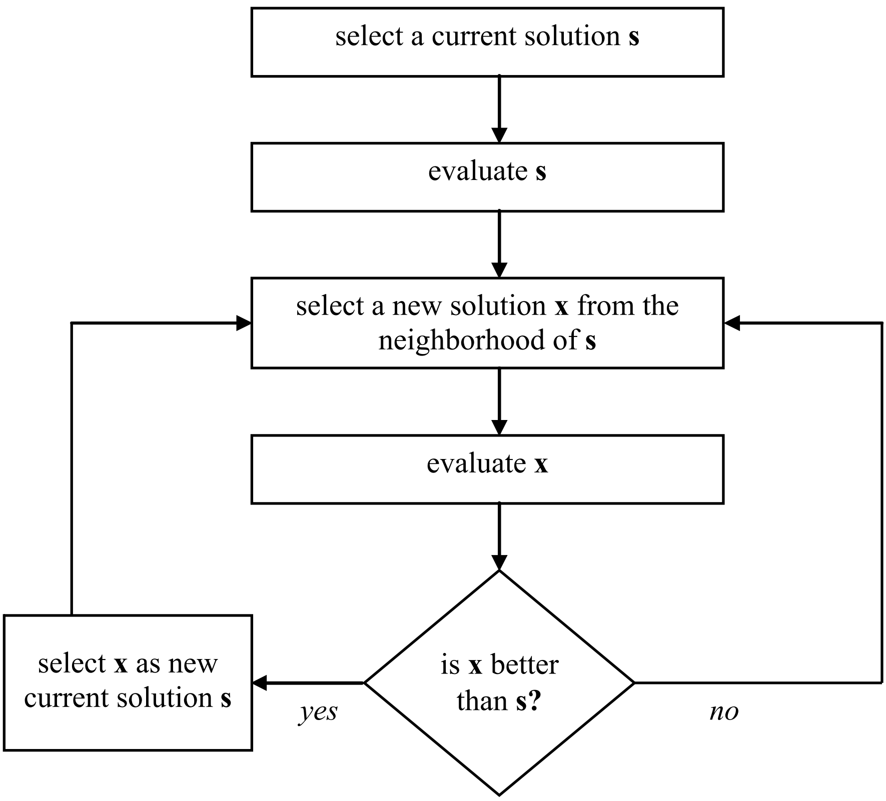
The evaluation function defines a *quality measure score landscape* (also known as a *response surface* or *fitness landscape*) that is much like a topography of hills and valleys. Within this three-dimensional landscape, the problem of finding a solution with the highest quality measure score is similar to searching for a peak in a foggy mountain range. Because our visibility is limited, we can only make local decisions about where to go next. If we always walk uphill, we will eventually reach a peak, but this peak might not be the highest peak in the mountain range; it might just be a "local" optimum. We may have to walk downhill for some period of time to find a path that will eventually lead us to the highest peak (i. e., the "global" optimum).

The quality measure score landscape for a two-variable function is illustrated below. The graph displays the quality measure score for every pair of values for the first and second variable, which allows us to visualize the mountain ranges, highest peaks, local optima, etc.:



Keeping this illustration in mind, let us examine a basic local optimization procedure called *hill climbing,*[23] and its connection with the "neighborhood" concept. Like all local optimization techniques, hill climbing uses iterative improvement. The technique is applied to a single solution (i. e., the current solution) in the search space. During each iteration, a new solution is selected from the neighborhood of the current solution. If that new solution has a better quality measure score, then the new solution becomes the current solution. Otherwise, some other neighbor is selected and tested against the current solution. The techniques terminates if no further improvements are possible, or when the allotted time runs out.

A simple flowchart of a hill-climbing sequence is given below:



Note that this flowchart expresses only the general principle of hill climbing without any termination conditions. We have to start with some (possibly randomly generated) solution **s**, evaluate it, and then generate a new solution **x** from

[23] The term *hill climbing* implies a maximization problem, but the equivalent *descent* method is easily envisioned for minimization problems. For convenience, the term will be used to describe both methods without any implied loss of generality.

a neighborhood of **s**. If the new solution **x** is better than **s**, then we take an uphill step (i. e., we accept this new solution as the current solution, and try to improve it further by generating yet another new solution from the neighborhood of the current one). On the other hand, if the new solution **x** is not better than **s**, we generate another new solution and we repeat this process several times until either (1) the whole neighborhood has been searched, or (2) we have exceeded the threshold of allowed attempts (which is missing from the flowchart). At this stage, we can exit the loop and report the current solution as the best solution, or we can store the current solution in "memory" and restart the whole process, hoping that the next hill-climbing iteration (which starts from a new solution) may produce a better overall solution (a process called *iterated hill-climbing*).

It is clear that such hill-climbing techniques can only provide locally optimum values that depend on the starting solution. Moreover, there is no general procedure for measuring the relative error with respect to the global optimum because it remains unknown. Given the problem of converging on locally optimal solutions, we often have to start the hill-climbing algorithm from a large variety of different solutions. The hope is that at least some of these initial locations have a path that leads to the global optimum. We might choose the initial solutions at random, or we might base them on some grid, regular pattern, or other available information (perhaps using the search results from somebody else's effort to solve the same problem).

The success or failure of a single iteration (i. e., one complete climb) of the hill-climbing algorithm is determined completely by the initial solution. For problems with many local optima, it is often very difficult to find the global optimum. Consequently, hill-climbing techniques have several weaknesses:

- They usually terminate at solutions that are only locally optimal.
- There is no information as to how much the discovered local optimum deviates from the global optimum, or perhaps even from other local optima.
- The optimum that is obtained depends on the initial configuration.
- In general, it is *not* possible to provide an upper bound for the computation time.

On the other hand, there is a tempting advantage to using hill-climbing techniques: they are very easy to apply! All that is needed is the representation, the evaluation function, and a measure that defines the neighborhood around a given solution.

Effective optimization techniques provide a mechanism for balancing two apparently conflicting objectives at the same time: *exploiting* the best solutions found so far, and *exploring* the search space. Hill-climbing techniques exploit the best available solution for possible improvement, but they neglect exploring a large portion of the search space. In contrast, a random search (where various solutions are sampled from the entire search space with equal probability) explores the search space thoroughly, but foregoes exploiting promising regions of the space. Each search space is different, and even identical spaces can appear very different under different representations and evaluation functions. As a result, there is no way to choose a single optimization technique that performs well in every case (more on this topic in Sect. 10.2).

Let us illustrate the hill-climbing technique on the car distribution example. Say we would like to implement an iterative hill-climbing algorithm that would generate a car distribution recommendation. Using the first representation (i. e., where a vector of 3,000 values provides indices of auction sites from 1 to 50) and defining a "neighbor" as a solution that differs (at most) by 1 on any position, the hill-climbing algorithm would work as follows.

First, the algorithm would generate a starting solution. This solution might be generated randomly (i. e., for each entry, a random number from 1 to 50 is produced) or we can accept some heuristic-based solution (e. g., an initial solution that assigns each car to the nearest auction site). Either way, let us assume that the initial solution is:

23	41	5	…	19	41

The algorithm then evaluates this solution and assigns a quality measure score to it. For this example, let us assume that the above solution generates a quality measure score of 171.49. Now we are ready to do some "hill-climbing"! The algorithm generates a neighbor solution by generating some random locations in the vector (any number of locations from 1 to 3,000) and then changing the selected indices in these locations by one (increment or decrement). Assume that the generated solution is (i. e., the selected first, second, …, and 3,000th location was increased or decreased by one.):

24	41	6	…	19	40

Next, the evaluation of this solution is needed. If the evaluation produces a quality measure score higher than the original solution (e. g., 176.18), then the algorithm will accept this new solution as the current solution and continue. Note that this new solution (with a higher quality measure score) has its own new neighborhood, and the subsequent new solution is drawn from this new neighborhood. Any acceptance of a new solution means that the algorithm found a better solution and made a step uphill. However, it may happen that the quality measure score of the new solution is lower than the current solutions (e. g., 169.83). In such a case, the algorithm will discard this solution (we are not interested in inferior solutions) and generate another solution from the neighborhood of the current solution. Say the next solution is:

24	42	5	…	18	40

Again, if there is an improvement in the quality measure score, then the algorithm will accept this solution and continue. If not, the algorithm will generate another solution from the original neighborhood …

Note also that a hill-climbing algorithm can (a) accept the first solution found that is better than the current one (as presented above), or (b) accept the best solution found in the whole neighborhood. These two possibilities represent two extremes, with plenty of "in between" possibilities (e. g., we can accept the best solution found from 100 generated solutions in the neighborhood).

The question is, how long should the hill-climbing algorithm generate random solutions before giving up? Well, we usually have a counter responsible for counting the algorithm's attempts to improve the current solution. Each time the algorithm finds an improvement the counter is reset to zero. However, if the hill-climbing algorithm experiences a long sequence of unsuccessful attempts, we stop the search upon exceeding a predefined threshold. In this particular example, what should the threshold be? The answer depends on a few factors, with the size of the neighborhood being the most important. It is difficult to claim that we have found the "local optimum" if we did not search the whole neighborhood, but the size of the neighborhood might be too large to evaluate all the neighbors! This problem can be resolved by defining a neighborhood differently. For example, if a neighbor differs from the current solution by only 1 on *one* location, then we will have up to 6,000 neighbors for each current solution and can evaluate all of them before giving up.

In summary, if it is feasible (time wise) to search the whole neighborhood before arriving at the local optimum, then we do not need a counter for controlling the number of unsuccessful attempts because *all* the solutions in the neighborhood will be searched. However, if it is not feasible to search the whole neighborhood, we have to settle for a counter and quit our search after some number of unsuccessful attempts. In our case, let us assume we quit the search after 100,000 unsuccessful attempts.

Returning to our example, say we arrive at the following solution after many iterations and improvements, and all attempts to improve it have failed:

27	31	9	…	45	29

Note the significant number of improvements the algorithm went through: the original assignment for car 2,999 (second to last position in the vector) changed from auction 19 to auction 45, and all changes were made by adding or subtracting 1. Anyway, in all likelihood we have arrived at the local optimum, and this solution is the outcome of our hill-climbing exercise. The quality measure score is 345.67 and we are confident about the solution's quality. After all, 100,000 neighboring solutions failed to produce any improvement!

However, we are not sure if this is the best solution. If we started our hill-climbing exercise from a different solution (which might be located in a very "different area" of the search space), we might finish with a local optimum solution that looks like:

43	41	32	…	15	27

and has a quality measure score of 1,457.81 (which is *much* better than the solution we discovered earlier!).

Recall our earlier discussion on the "hills and valleys" in a quality measure score landscape. Clearly, there are many hills (local optimum solutions) and the hill-climbing algorithm will produce a solution that represents one of these hills. However, the problem is that we do not know whether there are other (possibly much higher) hills somewhere else! And the size of the neighborhood corresponds to our "visibility" during the search: the larger the neighborhood, the better the visibility, and the better chances of discovering the highest peak! However, it might not be feasible to search the whole neighborhood if it is too large …

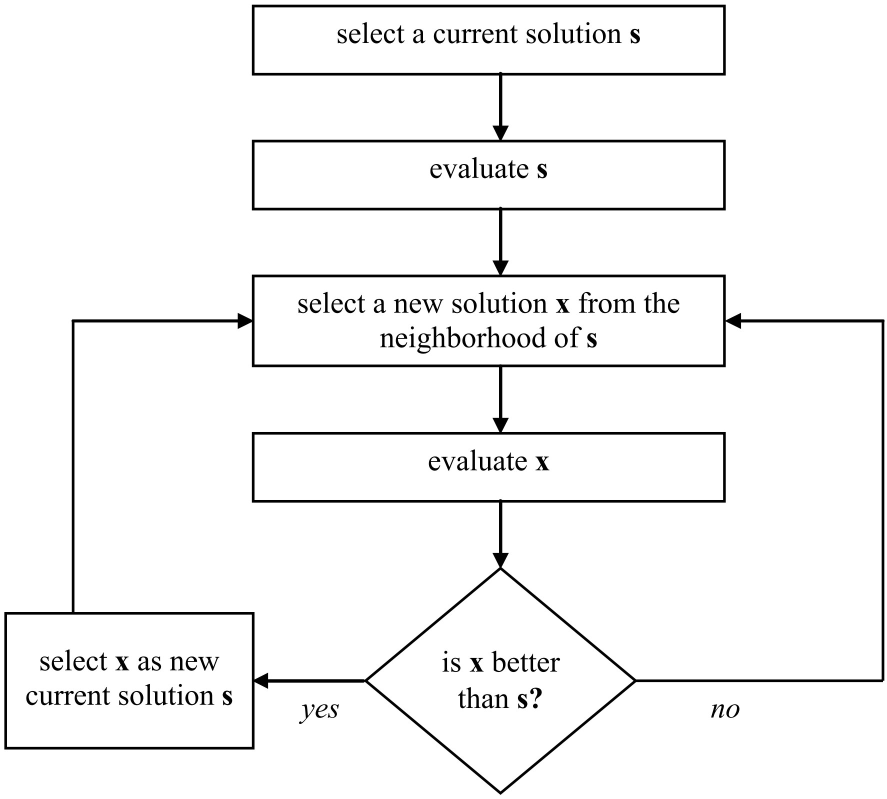
So, what should we do? We can restart out hill-climbing algorithm several times, each time from a different (possibly random) location, and hope that one of these runs will provide us with the global optimum solution (which may or may not happen).

6.3 Stochastic Hill Climber

Getting stuck in local optima is a serious problem. It is one of the main deficiencies of numerical optimization applications, as almost every solution to a real-world problem in factory scheduling, demand planning, land management, and so forth is at best only locally optimal.

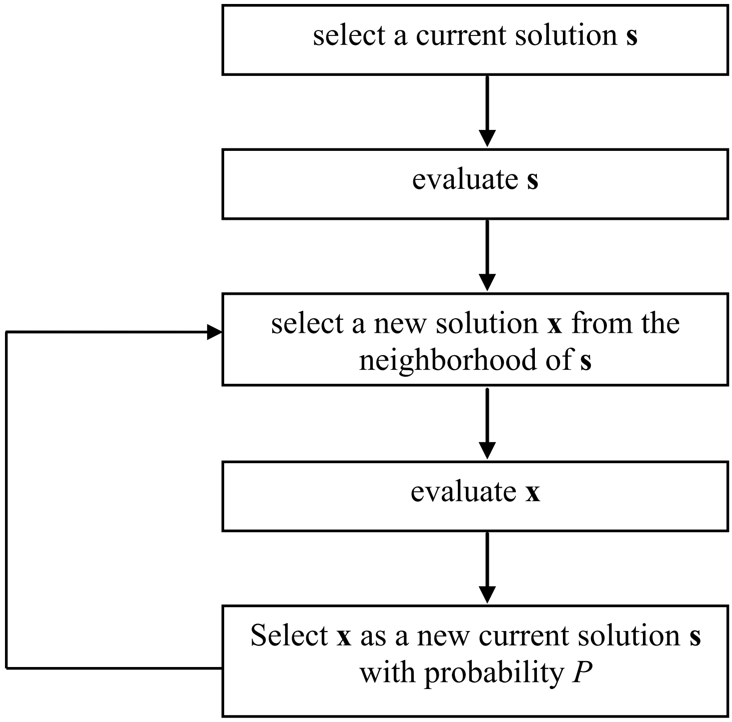
So what can we do about it? How can we design an optimization technique that has a chance to escape local optima, to balance exploration and exploitation, and to make the search independent from the initial configuration? There are a few possibilities, and we will discuss some of them in this chapter, but keep in mind that the proper choice is always dependent on the problem. One option, as we discussed earlier, is to execute a large number of initial configurations for the chosen technique. Moreover, it is often possible to use the results of previous attempts to improve the initial configuration for the next attempt. We have already seen one possibility of this in the previous section, where we discussed a procedure called the "iterated hill climber." After reaching a local optimum, the search is restarted from a different starting solution. Although we can apply this strategy to other algorithms, let us discuss some other possibilities of escaping local optima within a single run of an algorithm. One way of accomplishing this is by modifying the criteria for accepting new solutions that correspond to a *negative* change in the quality measure score. That is, we might want to accept an inferior solution from the local neighborhood in the hope that it will eventually lead us to something better.

To turn an ordinary hill climber into such an algorithm, a few modifications are required. First, let us recall the detailed structure of a hill climber:



Note again that the inner loop *always* returns the local optimum. The only way for this technique to "escape" local optima is by starting a new search (outer loop) from a new (random) location. After some maximum number of attempts, the best overall solution is the final outcome of the algorithm.

By modifying this procedure so that acceptance of a new solution is dependent upon some probability – which is based on the difference between the quality measure score for these two solutions – we obtain a new technique called the *stochastic hill climber*:



The slight (but significant) difference between an ordinary and stochastic hill climber lies in a single box inserted in the flowchart that replaces the condition box. During the execution of the hill climber's internal loop (where the hill-climbing searches for a better solution in the neighborhood of the current one), only a superior solution is accepted as a new current solution. On the other hand, the same internal loop in the stochastic hill climber procedure may accept an inferior solution as a new current solution. This feature does not appear in local optimization techniques. This insertion represents a probabilistic decision on the acceptance of a new solution (as opposed to a deterministic decision in classic hill climbers), and is done to escape local optima …

Let us discuss this feature carefully. A new solution **x** is accepted with some probability *P*, which means that the rule of moving from the current solution to a new neighbor is probabilistic. Consequently, it is possible for the newly accepted solution **x** to be *inferior* to the current solution **s**, and it is also possible that a superior solution will *not* be accepted! This probability of acceptance depends on the quality measure score difference between these two solutions, as well as on the value of an additional parameter *T* (which remains constant during the execution of the algorithm).

Rather than providing a mathematical function for calculating the values of probability P (which is based on a constant value of parameter T), we will instead explain how this function works. In general terms, the probability function is constructed in a such way that:

- If the new solution **x** has the same quality measure score as the current solution **s**, then the probability of acceptance is 50% (it does not matter which one is chosen, because each is of equal quality).
- If the new solution **x** is superior, then the probability of acceptance is greater than 50%. Moreover, the probability of acceptance grows together with the (negative) difference between these two quality measure scores.
- If the new solution **x** is inferior, then the probability of acceptance is smaller than 50%. Moreover, the probability of acceptance shrinks together with the (positive) difference between these two quality measure scores.

The probability of accepting a new solution **x** also depends on the value of parameter T, and the general principle is as follows:

- If the new solution **x** is superior, then the probability of acceptance is closer to 50% for *high* values of parameter T, or closer to 100% for *low* values of parameter T.
- If the new solution **x** is inferior, then the probability of acceptance is closer to 50% for *high* values of parameter T, or closer to 0% for *low* values of parameter T.

This is interesting, because it means that a superior solution **x** would have a probability of acceptance of *at least* 50% (regardless of the value of parameter T). Likewise, an inferior solution would have a probability of acceptance of *at most* 50% (varying between 0% for low values of T and 50% for high values of T). The general conclusion is clear: The lower the value of T, the more the algorithm behaves like a classic hill climber that rejects inferior solutions and accepts superior ones. On the other hand, if the value of T is very high, then the algorithm resembles a random search, because the probability of accepting inferior or superior solutions is close to 50%. Thus, we have to find a value for parameter T that is neither too low nor too high for a particular problem.

The stochastic hill climber technique is also a forerunner to another optimization technique called *simulated annealing,* which is covered in the next section.

6.4 Simulated Annealing

The simulated annealing technique (also known as Monte Carlo annealing, statistical cooling, probabilistic hill-climbing, stochastic relaxation, and the probabilistic exchange algorithm) is based on an analogy taken from thermodynamics. To grow a crystal, we begin by turning the raw material into a molten state through heating. Then we reduce the temperature of this crystal melt until the crystal structure is frozen. However, if the cooling process is done too quickly, then the results are detrimental. In particular, some irregularities are locked into the crystal structure and the trapped energy level is much higher than in a perfectly structured crystal.[24] The analogy between the physical system and an optimization problem is evident; the basic "equivalent" concepts are listed below:

- State – feasible solution
- Energy – evaluation function
- Ground state – optimal solution
- Rapid quenching – local search
- Temperature – control parameter T
- Careful annealing – simulated annealing

Simulated annealing is similar to a stochastic hill climber in that it may accept an inferior solution as a new current solution, and the acceptance decision is based on the value of parameter T. However, unlike the stochastic hill climber (which has a fixed value for parameter T), simulated annealing changes the value of parameter T (commonly referred to as *temperature*) during the run. Simulated annealing starts with high values of parameter T – making the process similar to a random search – and then gradually decreases this value during the run. The value of parameter T is quite small toward the end of the run, so the final stages of simulated annealing resemble an ordinary hill climber. Another difference between the stochastic hill climber and simulated annealing is that the latter always accepts superior solutions. Recall from the previous section that the stochastic hill climber used some probability for accepting both inferior *and* superior solutions, which is not the case in simulated annealing.

The following flowchart represents a simulated annealing algorithm:

[24] A similar problem occurs in metallurgy when heating and cooling metals.